

CSCE 614 COMPUTER ARCHITECTURE · MODULE 04 · LECTURE NOTES

Branch Prediction and Speculation

Guessing the future, and the cost of being wrong.

MODULE OUTCOMES

- Explain why branch prediction is necessary rather than merely helpful.
- Describe static, bimodal, two-level, and modern predictors.
- Measure the misprediction penalty on your own machine.
- Write branchless code and judge when it is worth it.
- Explain speculative execution and its security consequences.

1 Why prediction is mandatory

Module 03 identified control hazards as the expensive case. The arithmetic shows why they cannot simply be tolerated.

- Roughly one instruction in five is a branch — conditionals, loop back-edges, calls, returns.
- A modern pipeline is 15–20 stages deep, and a branch is not resolved until well into it.
- Stalling on every branch therefore contributes about $0.2 \times 15 = 3$ to CPI, taking an ideal CPI of 1 to roughly 4.

A fourfold slowdown is not acceptable, so processors guess which way each branch will go and continue executing immediately. Branch prediction is not a refinement that makes deep pipelines faster; it is the precondition that makes them possible at all.

1.1 Speculative execution

Having predicted, the processor fetches and executes down the predicted path, holding results provisionally rather than committing them to architectural state. If the prediction proves correct, the results commit and nothing was lost. If it proves wrong, all speculated work is discarded and fetching restarts at the correct address, costing roughly the pipeline depth.

At 95% accuracy the expected cost per branch is about $0.05 \times 15 = 0.75$ cycles instead of 15 — an overwhelming win, which is why the technique is universal despite its complexity and, as §4 discusses, its security cost.

2 How predictors work

Predictor	Mechanism	Typical accuracy
Static	A fixed rule: always predict taken, or predict backward branches taken and forward branches not taken (which approximates loops well).	60–70%
1-bit dynamic	One bit per branch recording the last outcome; predict the same again.	~80%
2-bit bimodal	A saturating counter per branch; two consecutive wrong outcomes are needed to change the prediction.	85–90%
Two-level / gshare	Index a table of counters by the branch address combined with a global history of recent outcomes, capturing correlation between branches.	93–96%
TAGE, perceptron	Several tables indexed by different history lengths, tagged to detect which applies, with the longest matching history winning.	98–99%

THE 2-BIT COUNTER, AND WHY HYSTERESIS MATTERS

Consider a loop executing 100 iterations, repeatedly.

A **1-bit** predictor mispredicts twice per execution: once on the final iteration (it predicts taken; the loop exits) and once on the first iteration of the next execution (it has now learned not-taken, but the loop is taken again).

A **2-bit saturating counter** requires two consecutive wrong outcomes to flip. The single loop-exit misprediction moves the counter but does not change the prediction, so the next execution starts correctly. One misprediction per execution instead of two.

One extra bit of state halves the misses. The idea — require corroboration before changing your mind — is simple and recurs throughout prediction and caching.

2.1 Correlation and two-level prediction

Branches are not independent events. Consider:

```
if (x > 0) { ... }           // branch A
if (x > 10) { ... }          // branch B -- highly correlated with A
```

If A was not taken, B cannot be taken. A predictor that considers each branch in isolation cannot exploit this; one that considers *recent global history* can.

A **two-level** predictor maintains a global history register of the last k branch outcomes and indexes its counter table by a combination of the branch address and that history. The same static branch then receives different predictions in different dynamic contexts. **gshare** uses the exclusive-or of the program counter and the global history as the index — simple to build, and a substantial accuracy improvement.

2.2 Three separate prediction problems

Question	Mechanism	Difficulty
Taken or not?	The direction predictor described above.	Largely solved: 98%+ on typical code.
Taken to <i>where</i> ?	Branch Target Buffer, caching recent targets by branch address.	Easy for direct branches (the target is encoded). Hard for indirect branches — virtual calls, function pointers, switch tables — where the target varies at run time.
Where does this return go?	Return Address Stack: push on call, pop on return.	Nearly perfect, because calls and returns nest. Deep recursion can overflow it, at which point accuracy collapses.

The practical consequence is that virtual dispatch is cheap when the target is stable — the BTB predicts it correctly — and expensive when it genuinely varies, which is precisely the situation in which polymorphism is doing useful work. This is why devirtualisation and type-stability matter in hot loops.

3 Measuring and avoiding

3.1 The sorted-array experiment

The clearest demonstration of branch prediction's cost requires changing nothing except predictability.

```
for (i = 0; i < N; i++) a[i] = rand() % 256;

// Loop 1: data is random
for (rep = 0; rep < 1000; rep++)
    for (i = 0; i < N; i++)
        if (a[i] >= 128) sum += a[i];

sort(a, N);

// Loop 2: identical code, identical data, now sorted
```

The two loops execute the same instructions on the same values with the same memory access pattern. The second is typically three to six times faster. The only difference is that after sorting, the branch is not-taken for the first half of the array and taken for the second, so the predictor is wrong roughly once instead of half the time.

Confirm it with `perf stat -e branch-misses`: the random version shows a miss rate near 50%, the sorted version near 0%.

3.2 Branchless code

```
// Branchy: 0 cycles when predicted, ~15 when not
if (a[i] >= 128) sum += a[i];

// Branchless: always 1-2 cycles, never mispredicts
sum += a[i] * (a[i] >= 128);

// Conditional move -- often generated automatically
int t = (a[i] >= 128) ? a[i] : 0;
sum += t;
```

BRANCHLESS IS A DIFFERENT RISK PROFILE, NOT A STRICT IMPROVEMENT

A branch that predicts well is essentially free. Replacing it with arithmetic means paying a small cost on every iteration instead of a large cost almost never — which is a net loss.

A branch that is genuinely unpredictable costs the full misprediction penalty about half the time. Replacing it is a large win.

So the question is never 'is branchless faster?' but 'is this branch predictable?'. Measure the branch miss rate before deciding, and measure the result afterwards — compilers also convert branches to conditional moves on their own, sometimes wrongly.

Situation	Go branchless?
Data-dependent branch, roughly 50/50	Yes — the clearest win.
Branch taken 99% of the time	No — prediction already costs nothing.
Both sides cheap and side-effect free	Probably yes.
One side expensive	No — you would execute it every time.
One side has side effects or may fault	No — branchless evaluates both.
Hot inner loop	Measure. This is not a case for intuition.

4 Speculation and security

SPECTRE

Speculatively executed instructions are discarded architecturally when the prediction is wrong. But they *ran*, and their memory accesses left lines in the cache.

An attacker trains the predictor so that a bounds check is speculatively bypassed, causing a read of out-of-bounds memory and then a second access indexed by the secret value. The architectural result is discarded; the cache state is not. Timing subsequent accesses reveals which line was loaded, and hence the secret.

The ISA promised that mis-speculated instructions have no architectural effect, and that promise was kept. It said nothing about microarchitectural state or timing, and those were sufficient.

This is the abstraction leak of Module 02 in a form nobody designed for. Mitigations — speculation barriers, indirect branch restrictions, and various flushes — cost real performance, in some workloads 5–30%. The episode is a reminder that an abstraction which is only approximately true provides a security guarantee only by accident.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Onur Mutlu — Branch Prediction lectures	Static through TAGE, with the correlation insight developed properly.
Dan Luu — Branch prediction	A short, clear history of predictor designs with the intuition for each.
Stack Overflow — 'Why is processing a sorted array faster?'	The canonical demonstration, with excellent answers. Run it yourself before reading.

Resource	What it covers for this module
Spectre Attacks: Exploiting Speculative Execution (free paper)	The original paper. Readable, and worth reading in full for how carefully the abstraction is picked apart.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Run the sorted-array experiment. Measure both versions and confirm with `perf stat -e branches,branch-misses` that the difference is misprediction and not something else.
- 2 Derive your machine's misprediction penalty in cycles: construct a loop with a perfectly predictable branch and one with a random branch, matched for instruction count, and solve for the penalty.
- 3 Rewrite the branchy loop branchlessly. Measure it on random and on sorted data, and explain why it wins in one case and loses in the other.
- 4 Write a loop whose branch pattern is periodic with period 4 (TNTN...). Measure the miss rate and explain why a modern predictor handles it nearly perfectly.
- 5 Measure an indirect call through a function pointer that is stable, versus one that alternates between two targets, versus one chosen at random from eight. Explain the pattern in terms of the BTB.
- 6 Write a deeply recursive function and measure where return prediction degrades. Relate the depth to the likely Return Address Stack size.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Compute the CPI cost of stalling on every branch with 20% branches and a 15-stage pipeline. Why does this make prediction mandatory?
- 2 Why does a 2-bit predictor outperform a 1-bit predictor on loops? Be specific about which mispredictions it removes.
- 3 What does global history let a predictor exploit that a per-branch predictor cannot?
- 4 Name the three separate things that must be predicted at a branch, and which is hardest.
- 5 Why is a sorted array faster to process, given identical instructions and identical memory traffic?
- 6 Give two situations where branchless code is a loss.
- 7 What exactly did Spectre exploit, and which guarantee was *not* violated?